

Mobile Application Development

Android Intent

Today's Session

- Intent
 - Implicit Intent
 - Explicit Intent
 - Passing data between activities
- Menu
 - Option menu
 - Context menu
- Lab work: create applications with more than 1 activity and menu

What is Intent

- An intent is an object that describes something that needs to be accomplished and is sent from one component of an application to another component.
- Its most significant use is in the launching of activities, where it can be thought of as the glue between activities.
- Intent carries the action to be performed, and the necessary information required to perform the intended action.

Components using Intent

- Android components that can process Intent objects include Activities, Services, and Broadcast Receivers
- Intent is used to
 - launch another activity
 - to start a service or communicate with a background service, e.g.: start music player
 - Broadcast Intent for broadcasting communication message to any interested Broadcast Receiver component, e.g.: sending intent when change time zone, any broadcast receivers registered this event will get the intent.

Intent data structure

- An intent can include a component name (target receives intent), action (that the target needs to perform), data, type, categories and extra.
- The **component name** of the component (class) which needs to process the intent (optional).
- The **action** that needs to be performed.
 - Android provides a list of actions (constant) that we can use, e.g.:
 - ACTION_DIAL (Display a phone number to dial)
 - ACTION_CALL (Make a phone call)
 - ACTION_MAIN (Start the main activity for the app)
 - ACTION_EDIT (Edit a database)
 - ACTION_VIEW (View the content of a URI)
 - ACTION_WEB_SEARCH (Search on the web with a search string)
 - etc.

Intent data structure

- The **data** that the action needs to operate on
 - the data reference points to the data that the action is to act upon.
 - data reference is in form of URI (Universal Resource Identifier).
- The **type** of data that is being processed e.g. text/plain-text, video/mp4, audio/mp3, etc.
 - When an element of Intent is left out, Android will infer what should go in that element.

Intent data structure

- The **category**
 - A string containing additional information about the action to execute. E.g.:
 - **CATEGORY_LAUNCHER** : Using with ACTION_MAIN to launch the target activity as the initial task.
 - **CATEGORY_APP_BROWSER**: Using with ACTION_MAIN to launch the browser app
- **Extras**
 - Additional information needed to send over with intent, in form of key-value pair.

The most common elements used are component name, action, data and extras

Intent class

- public class Intent
- To use intent, we create an object of Intent class, then define components of intent according to the data structure
- Call startActivity with the intent object.

Explicit and Implicit Intents

- Two types of Intent: Explicit and Implicit intents
- Explicit Intent
 - Specify the component to start by indicating component name
 - Typically used to start a component in your own app because you know its name
 - E.g., you tell your app to move to another screen (start another activity)
- Implicit Intent
 - Do not name the component but instead declare a general action to perform and allows a component from another app to handle it
 - E.g., display a location on a map using some mapping app, searching on a web browser but not indicating the browser name.

Implicit intent ACTION_VIEW

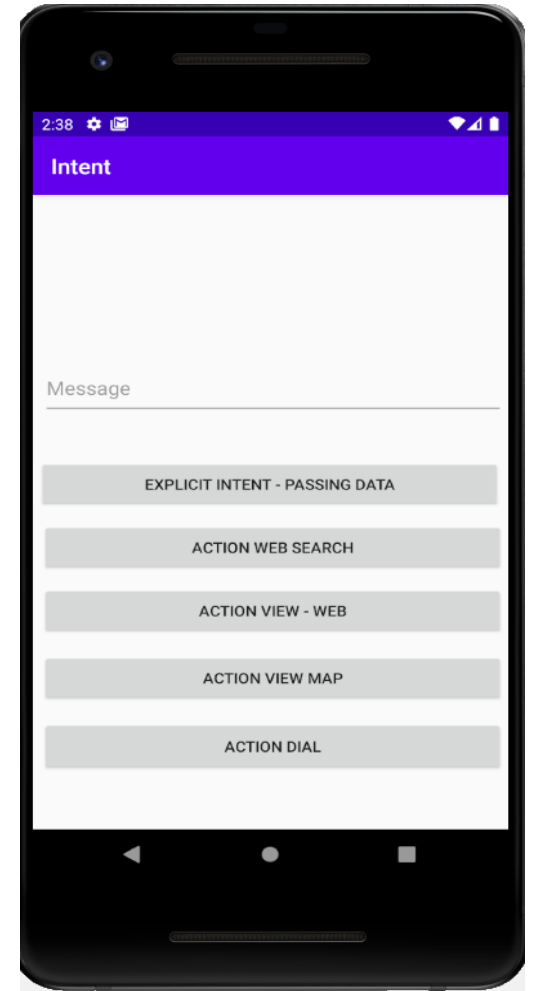
- Used as implicit activity, we intent to view data, Android system provides suitable app to view that data
- For instance, implicit intent to view a webpage, including **action** and **data**, data is in form of uri:

```
//This code is inside onCreate() method

Button btn3 = (Button) findViewById(R.id.button3);
btn3.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {

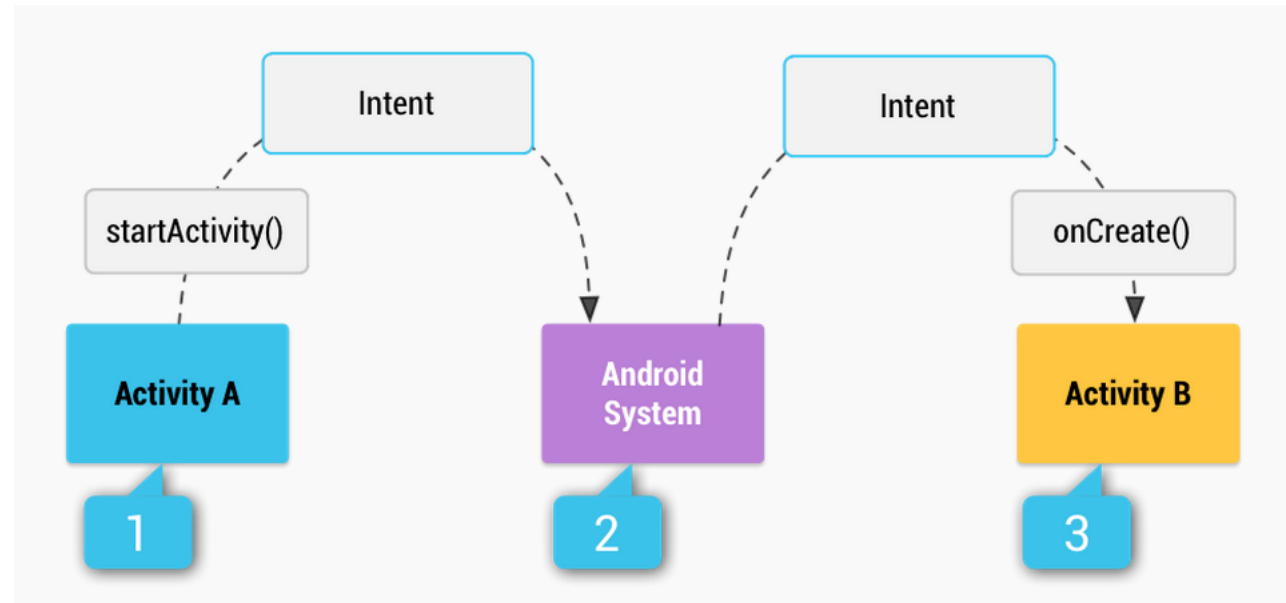
        Intent i = new Intent(Intent.ACTION_VIEW,
Uri.parse("https://www.irishtimes.com/"));
        startActivity(i);

    }
});
```



Implicit Intent

- The Android system finds the activity capable of handling the intent created by Activity A



[1] Activity A creates an Intent and call `startActivity()` with that intent. [2] The Android System searches all apps for an intent filter that matches the intent. When a match is found, [3] the system starts the matching activity (Activity B) by invoking its `onCreate()` method and passing it the Intent

ACTION_WEB_SEARCH

- Implicit intent to search on the web

```
Intent intent = new Intent(Intent.ACTION_WEB_SEARCH);  
intent.putExtra(SearchManager.QUERY, "straight hitting  
golf clubs");  
startActivity(intent);
```

- The web browser is not mentioned, Android will pick the best matched, or we can explicitly ask user to pick the apps that can handle intent by calling `createChooser`

```
startActivity(Intent.createChooser(intent, "Choose the app"));
```

ACTION_VIEW

- implicit intent to view the list of contacts on your phone

```
Intent i = new Intent(Intent.ACTION_VIEW,  
Uri.parse("content://contacts/people/"));  
startActivity(i);
```

- A content:// URI indicates the data is located on the device and controlled by a Content Provider
- Try this:
 - Input some contacts information in the emulator then try to view contact list

ACTION_VIEW

- Implicit intent to view the first contact from the contact list

```
Intent i = new Intent(Intent.ACTION_VIEW,  
Uri.parse("content://contacts/people/1"));  
startActivity(i);
```

ACTION_VIEW

- Implicit intent to view location on Google map

//geo: latitude, longitude, z is zoom level

```
Uri location = Uri.parse("geo:53.3385,-6.2666?z=15"); // z param is zoom level
Intent mapIntent = new Intent(Intent.ACTION_VIEW, location);
startActivity(mapIntent);
```

The following list shows the approximate level of detail you can expect to see at each zoom level:

1: World

5: Landmark/continent

10: City

15: Streets

20: Buildings

Finding geocoding: <http://www.gpsvisualizer.com/geocode>

Intent Filters

- While an intent is effectively a request to do something, an intent filter is a description of what intents an activity (or intent receiver) is capable of handling
- Activities publish their IntentFilters in the AndroidManifest.xml file.
- Each intent filter specifies the type of intents it accepts based on the intent's action, data, and category. The system delivers an implicit intent to your app component only if the intent can pass through one of your intent filters.
- Navigating from screen to screen is accomplished by resolving intents.

Intent Filters

- To navigate forward, an activity calls `startActivity(myIntent)`.
- The system then looks at the intent filters for all installed applications and picks the activity whose intent filters best matches `myIntent`.
- The new activity is informed of the intent, which causes it to be launched.

Intent Filter in Manifest file - Example

- Use the `<intent-filter>` tag

```
<activity android:name="MainActivity">
  <!-- This activity is the main entry, should appear in app launcher -->
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>
```

```
<activity android:name="ShareActivity">
  <!-- This activity handles "SEND" actions with text data -->
  <intent-filter>
    <action android:name="android.intent.action.SEND"/>
    <category android:name="android.intent.category.DEFAULT"/>
    <data android:mimeType="text/plain"/>
  </intent-filter>
</activity>
```

Explicit intent

- Indicate what component/activity you want to start using the component/activity name.
- Create an intent object with the class name of the activity you want to start
 - `Intent intent = new Intent(context, target activity name)`
Context is the context where the intent is created and sent, usually use `Activityname.this`
- Call `startActivity` method with that intent

Explicit Intent

- Call to start another activity
- For instance, ActivityOne call to start ActivityTwo, using this code:

```
# Start the activity connect to the  
# specified class  
  
Intent i = new Intent(this, ActivityTwo.class);  
startActivity(i);
```

Where **this** is the context (context is the current activity or explicitly indicating the context using activity name.this), which refers to the activity sending the intent.

Or this code:

```
Intent i = new Intent(ActivityOne.this, ActivityTwo.class);  
startActivity(i);
```

Explicit Intent – sending data – putExtra()

- An Intent can carry a collection of various data types as key-value pairs called *extras*
- We can save data to intent in one activity, and retrieve data in intent from another activity, this data is saved in Bundle object.
- The `putExtra(key, value)` method to save data which is in pair (key, value)

key is string, *value* can be int, string, Boolean, float, Array, etc.

- In order for the next activity to query the extra data, you should define your key using a public constant as it is used in other activity.

e.g.: `public static final String KEY1 = "MESSAGE1";`

Put data in intent

- Example, we want to pass data captured from an edit text whose id is editTextName from the MainActivity to ActivityTwo when clicking on a button in MainActivity
 - Inside onCreate() method of MainActivity, find the button using findViewById
 - Set onClick listener on this button, inside this method
 - Retrieve the edit text editTextName and capture its value
 - Create an intent object and call putExtra method
- Note that we can put multiple data using multiple putExtra.

Sending data in intent and call other activity

```
//This constant is defined at top of class
public static final String MESSAGE_KEY="MESSAGE1";

// this code is inside onCreate()

Button btn = (Button) findViewById(R.id.button);
btn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {

        EditText editTextname = (EditText) findViewById(R.id.editTextName);
        String name = editTextname.getText().toString();
        Intent intent = new Intent(MainActivity.this, ActivityTwo.class);

        intent.putExtra(MESSAGE_KEY, name);
        startActivity(intent);
    }
});
```

Receive the Intent – getExtra()

- In the target activity, we can get the Intent by calling getIntent() and retrieve the data contained within it.
- To retrieve data using getStringExtra(key) method, or getIntExtra(key, defaultvalue), or getBooleanExtra(key, defaultvalue), etc.
 - Make sure you use correct function depending on data sent, the best way is using getStringExtra and put String data in intent.
 - If there is defaultvalue in get method, if the key is not found, the defaultvalue is returned.

```
Intent intent = getIntent();
```

```
String message1 = intent.getStringExtra(MainActivity.MESSAGE_KEY)
```

```
// remember MESSAGE_KEY is a public static constant defined in the Main activity
```


Receive the intent

- ActivityTwo receives the intent, get data sent from activity one and display it in an edit text whose id is *editText*
- In onCreate() method of ActivityTwo, we have this code

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_two);

    Intent intent = getIntent();
    String name = intent.getStringExtra(MESSAGE_KEY);
    EditText edittxt = (EditText) findViewById(R.id.editText);
    edittxt.setText(name);
}
```

Other example of working with
more than 1 activity

Explicit Intent

Example

- Take a simple example, create a project name IntentTest
- In this example, we create 1 main activity to take in a message
- The Edit text contains the message which has its id is *message*
- 1 button to send the message
- Create another activity name DisplayMessageActivity, in this activity there is simply a text view which displays the message passed

Starting an Activity via an Intent

- Need to handle the button click event
- Create the method `sendMessage` in `MainActivity.java` class and define `onClick="sendMessage"` in the button item in the layout file

```
public void sendMessage(View v)
```

- Or using `setOnClickListener` on the button

Add Intent

- Inside the `sendMessage(View view)` method, create an Intent to start the activity `DisplayMessageActivity`

```
Intent intent = new Intent(this, DisplayMessageActivity.class);
```

Add Data to Intent and startActivity

- Use findViewById() to get the EditText element containing the message and add its text value to the intent using putExtra

```
Intent intent = new Intent(MainActivity.this, DisplayMessageActivity.class);  
EditText editText = (EditText) findViewById(R.id.message);  
String msg = editText.getText().toString();  
intent.putExtra(MESSAGE_KEY, msg); //MESSAGE_KEY is a constant defined  
in MainActivity class - static final  
startActivity(intent);
```

Full method

/** Called when the user clicks the Send button */

```
public void sendMessage(View view) {  
    Intent intent = new Intent(this, DisplayMessage.class);  
    EditText editText = findViewById(R.id.editTxtmessage); // suppose  
    // you set id of the edit text is editTxtmessage  
    String message = editText.getText().toString();  
    intent.putExtra(MESSAGE_KEY, message);  
    startActivity(intent);  
}
```

Receive the Intent in DisplayMessage activity

- Get the Intent that started your activity by calling `getIntent()` and retrieve the data contained within it.
- In the `DisplayMessageActivity onCreate()` method, get the intent and extract the message delivered by the first Activity

```
Intent intent = getIntent();
```

```
String message = intent.getStringExtra(MainActivity.MESSAGE_KEY);
```


Display the message – the target activity

- To show the message on the screen
 - Display the message on suitable text view

```
public void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
  
    ...  
  
    // Get the message from the intent  
    Intent intent = getIntent();  
    String message = intent.getStringExtra(MainActivity.MESSAGE_KEY);  
    //find the text view using findViewById, e.g. its id is textViewMsg  
    TextView resulttext = findViewById(R.id.textViewMsg)  
    resulttext.setText(message);  
}
```

Summary

- Intent object allows to perform actions across components of an application or between applications
- There are Implicit intent and Explicit intent
- Explicit intent is used to start other activities that the name is known
- It is possible to pass data to other activities with intent

More references

<https://developer.android.com/training/basics/firstapp/building-ui.html>

<https://developer.android.com/training/basics/firstapp/starting-activity.html>

More on intent:

<https://developer.android.com/reference/android/content/Intent.html>